

Your grade: 100%

Your latest: 100% • Your highest: 100% • To pass you need at least 80%. We keep your highest score.

Next item →

1. In the context of implementing the Wave Function Collapse algorithm for procedural generation, managing tile compatibility is essential for ensuring that generated patterns are coherent. Which data structure correctly represents a way to store compatibility information for a single tile, using keys for each tile edge ('top', 'bottom', 'left', 'right') and string values to denote compatible types?
- 1 / 1 point

- ☐

```
1 tile_compatibility = {
2     "grass",
3     "water",
4     "mountain",
5     "forest"
6 }
```
- ☐

```
1 tile_compatibility = {
2     1: "grass",
3     2: "water",
4     3: "mountain",
5     4: "forest"
6 }
```
- ☒

```
1 tile_compatibility = {
2     "top": "grass",
3     "bottom": "water",
4     "left": "mountain",
5     "right": "forest"
6 }
```
- ☐

```
1 tile_compatibility = {
2     ["top", "grass"],
3     ["bottom", "water"],
4     ["left", "mountain"],
5     ["right", "forest"]
6 }
```
- ☐

```
1 tile_compatibility = "top-grass-bottom-water-left-mountain-right-forest"
```

☒ **Correct**
Correct. This dictionary correctly uses 'top', 'bottom', 'left', 'right' as keys, each associated with a string value that denotes the type of tile compatible with that edge. This structure allows for clear and direct access to compatibility information for each side of the tile, making it the optimal choice for managing edge compatibilities in a procedural generation context.

2. In the Wave Function Collapse (WFC) algorithm, the propagation process is crucial for updating the states of neighboring cells based on the constraints of already collapsed cells, thereby ensuring the generated pattern adheres to the specified rules. A simplified version of this process might involve propagating changes towards the cell with the lowest entropy (i.e., the cell with the fewest remaining possible states). Among the following code snippets, which one best represents a simplified approach to propagate updates in the WFC algorithm, focusing on cells with the lowest entropy?
- 1 / 1 point

- ☐

```
1 def propagate(grid):
2     for cell in grid:
3         if not cell.collapsed:
4             neighbors = get_random_neighbors(cell)
5             for neighbor in neighbors:
6                 update_cell(neighbor)
```
- ☐

```
1 def propagate(grid):
2     for cell in grid:
3         neighbors = get_all_neighbors(cell)
4         for neighbor in neighbors:
5             update_cell(neighbor)
```
- ☒

```
1 def propagate(grid):
2     next_cell = get_cell_with_lowest_entropy(grid)
3     if next_cell:
4         collapse_cell(next_cell)
5         neighbors = get_neighbors(next_cell)
6         for neighbor in neighbors:
7             update_cell(neighbor)
```
- ☐

```
1 def propagate(grid):
2     next_cell = get_cell_with_highest_entropy(grid)
3     if next_cell:
4         collapse_cell(next_cell)
5         neighbors = get_neighbors(next_cell)
6         for neighbor in neighbors:
7             update_cell(neighbor)
```
- ☐

```
1 def propagate(grid, external_conditions):
2     for cell in grid:
3         if meets_external_conditions(cell, external_conditions):
4             collapse_cell(cell)
5             neighbors = get_neighbors(cell)
6             for neighbor in neighbors:
7                 update_cell(neighbor)
```

☒ **Correct**
Correct. This code snippet effectively captures the essence of propagating changes towards the cell with the lowest entropy. It identifies and collapses the cell with the fewest possible states first, then updates its neighbors, aligning with the strategic approach of WFC to reduce uncertainty and ensure consistency across the grid.

3. In the implementation of the Wave Function Collapse (WFC) algorithm, collapsing a cell is a crucial step that involves selecting a state (or tile) from the set of possible states the cell can assume, thereby reducing its possibilities to one definite state. Which of the following code snippets correctly implements the process of collapsing a cell according to the WFC algorithm as described?
- 1 / 1 point

- ☐

```
1 def collapse_cell(self, x, y):
2     self.cells[x][y] = []
3     self.propagate(x, y)
```
- ☒

```
1 def collapse_cell(self, x, y):
2     possibilities = self.cells[x][y]
3     chosen_tile = random.choice(possibilities)
4     self.cells[x][y] = [chosen_tile]
5     self.propagate(x, y)
```
- ☐

```
1 def collapse_cell(self, x, y):
2     self.cells[x][y] = ['fixed_state']
3     self.propagate(x, y)
```
- ☐

```
1 def collapse_cell(self, x, y):
2     self.propagate(x, y)
```
- ☐

```
1 def collapse_cell(self, x, y):
2     chosen_tile = self.cells[x][y][0]
3     self.cells[x][y] = [chosen_tile]
4     self.propagate(x, y)
```

☒ **Correct**
Correct. This snippet accurately implements the cell collapse process in WFC by randomly selecting one state from the cell's set of possibilities and updating the cell to reflect this chosen state exclusively. Following the selection, it calls a propagate function to update adjacent cells, adhering to the procedure outlined in the reference code.